c267c0d0-9775-44a2-8ed0-c5382b782ab1

<prev_user>

<request>

You have been given comments on the previous commits you made in the repository.

You are working on an issue in the 'PyThaiNLP/pythainlp' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/pythainlp/pythainlp (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2026-01-07T08:47:14.341Z</current_datetime>

Consider the following problem statement:

<problem_statement>

This section includes details on the pull request with code changes you have already submitted to fix the problem above.

<pr_title>
Add Complete Soundex for Thai Words Similarity Analysis
</pr_title>

<pr_description>
What does this changes

Implements Complete Soundex algorithm for Thai syllable phonetic encoding based on Chalermpol Tapsai, Phayung Meesad, and Choochart Haruechaiyasak. 2020. Complete Soundex for Thai Words Similarity Analysis. Information Technology Journal KMUTNB. 2020 June 30;16(1):46–59. paper. Pure algorithmic approach without hardcoded overrides. **Designed to work on individual Thai syllables - users must tokenize multi-syllable words first.**

Usage:
```python
from pythainlp.soundex import complete_soundex
from pythainlp.tokenize import syllable_tokenize

# Single syllable encoding
complete_soundex("□□□□")  # '□□1B□2-'
complete_soundex("□□□□")  # '□□1A□0□'

# For multi-syllable words, tokenize first
syllables = syllable_tokenize("□□□□□")  # ['□□□', '□□']
codes = [complete_soundex(syl) for syl in syllables]
result = ''.join(codes)  # '□□4G0□-□□1B0--*'

# Via soundex engine parameter
from pythainlp.soundex import soundex
soundex("□□□□", engine="complete_soundex")  # '□□1B□0-'
```

What was wrong

PyThaiNLP lacked the Complete Soundex algorithm for Thai phonetic similarity analysis despite its proven effectiveness in Thai NLP applications. Additionally, the initial implementation had bugs in format detection and incorrectly handled syllable tokenization internally. The main processing method also had high cyclomatic complexity, making it difficult to maintain.

How this fixes it

Core Implementation:
- Initial consonant mapping with phonetic grouping (Table 5.1): 30+ consonants → 20 sound groups
- Vowel detection and encoding (Table 5.2): 15+ patterns including compound vowels (□□□□, □□□□)
- Final consonant normalization (Table 5.3): sonorant clustering rules
- Tone mark encoding (Table 5.4): 4 tone levels
- Special format for □/□/□-initial syllables: tone-final position swap with asterisk marker for □

Key Features:
- **Syllable-level encoding**: Works on individual syllables without internal tokenization
- Context-aware cluster detection (□/□/□) distinguishes phonetic clusters from syllable structure
- Syllable splitting heuristics handle multi-consonant patterns

(□□□□□□ → □□□+□□□)
- Special initial patterns (□□ → □ sound)
- Dropped □ in final clusters with formatting (□□□□ → □□1B-□0-)
- Pattern markers for □□/□□/□□ compound syllables and □-initial syllables
- Special format rules:
  - Initial consonants □, □, or □ trigger tone-before-final format
  - Final consonant □ or □ triggers special format
  - Final consonant □ with short vowel (1A) triggers special format

Bug Fixes:
- Fixed format detection for syllables ending in □ with short vowels (□□□ → □□1A0□-)
- Fixed format for syllables starting with □ (□□ → □□1B0--)
- Removed internal syllable_tokenize dependency to allow proper syllable-level encoding

Code Quality Improvements:
- Refactored complex `process_syllable` method into 10 focused helper methods to reduce cyclomatic complexity
- Each helper method has a single, clear responsibility for better maintainability
- Improved code readability while maintaining all functionality

Documentation:
- Module-level docstring with usage examples for single and multi-syllable words
- Function docstrings with tokenization guidance
- Test examples demonstrating recommended workflow with `word_tokenize()` and `syllable_tokenize()`

Important Note:
This soundex is designed to work on **individual Thai syllables**. For multi-syllable words, users should:
1. Tokenize text into words using `word_tokenize()`
2. Tokenize each word into syllables using `syllable_tokenize()`
3. Apply `complete_soundex()` to each syllable separately

Your checklist for this pull request

- [x] Passed code styles and structures
- [x] Passed code linting checks and unit test
</pr_description>

Comments on the PR (you are @copilot in this section)

<comments>

<pr_comments>

tokenize.syllable_tokenize and test set for this soundex that
want pythainlp.tokenize.syllable_tokenize
3. Add doc that should use pythainlp.tokenize.word_tokenize and
pythainlp.tokenize.syllable_tokenize before using this soundex
</comment_old>

<comment_old>
<author>@copilot</author>

> @copilot
>
> 1. I found a bug.
...

All three issues addressed in commit 13ed201:

1. **Bug fixed**: `cs.encode("□□□")+cs.encode("□□")` now
correctly produces `□□1A0□-□□1B0--`
   - Added special format rules for □ as initial consonant and □
as final with short vowel (1A)

2. **Removed syllable_tokenize dependency**: The `encode()`
method now works on individual syllables without internal
tokenization
   - Users should pre-tokenize multi-syllable words using
`syllable_tokenize()` before applying soundex

3. **Added documentation**:
   - Module-level docstring with usage examples showing both
single and multi-syllable word handling
   - Function docstrings updated with tokenization guidance
   - Test comments demonstrating the recommended workflow

Example usage is now documented in the code and tests.
</comment_old>

<comment_old>
<author>@bact</author>

```
@copilot Ruff reports too complex code at line 139 of pythainlp/
soundex/complete_soundex.py
</comment_old>

<comment_old>
<author>@copilot</author>

&gt; @copilot Ruff reports too complex code at line 139 of
pythainlp/soundex/complete_soundex.py

Fixed in commit 4c56fcc. Refactored `process_syllable` by
extracting 10 helper methods to reduce cyclomatic complexity.
The method went from 182 lines with 53 if statements to a clean
orchestration function that delegates to focused helper methods,
each with a single responsibility.
</comment_old>
```

```
<comment_new>
<comment_id>3717864774</comment_id>
<author>@bact</author>
@copilot Codacy static code analysis reports issues, see
https://app.codacy.com/gh/PyThaiNLP/pythainlp/pull-requests/
1175/issues#issue-39b9b93e2424ee09298c846e9b3f26db

- There are trailing whitespaces in the code

Make sure to format the code properly, using black.

Also do not use non-ASCII characters in function or variable
names,
as in `_adjust_□_mapping`.
</comment_new>
```

```
</pr_comments>

</comments>

----
The last **9** git commits in this branch are the changes you
have made so far. Use those as your change commit history.
</problem_statement>


Analyze and determine if action or explanation is needed
<rules_for_making_changes>
* Start by fully understanding <problem_statement> and the
<comments> before making any changes. You will only address the
<comment_new> comments. Reply to new comments as you address them.
```

Ignore comments that are not actionable or directed at you.
* For each new comment, determine:
    - If it a request for changes, a question on the code or a suggestion.
    - Is it addressed to you or someone else, e.g. "Hey @username, ..." or "@username can you ...".
    - Is it a general comment or praise.
* If there are no comments you need to address or reply to, you can stop now without exploring or explaining yourself.
* If the user's ask is not clear, you can reply to the comment and ask for clarification.
* If an action or explanation is needed, follow the steps below.
</rules_for_making_changes>

A **CI/Build Failure** has been mentioned in the problem statement, make sure to follow your instructions for addressing CI/Build Failures.

<prev_assistant>

Perfect! Let me create a final summary.

Summary

I successfully addressed the Codacy static code analysis issues reported in the comment.

Issues Identified
1. **Trailing whitespace**: Found on lines 9 and 11 in the module docstring (after author names and year)
2. **Non-ASCII characters in function name**: The function `_adjust_□_mapping` used the Thai character □
3. **Code formatting**: Need to format with black

Changes Made

1. **Removed trailing whitespace** from the docstring:
   - Line 9: After "Chalermpol Tapsai, Phayung Meesad, and Choochart Haruechaiyasak. 2020."
   - Line 11: After "Information Technology Journal KMUTNB. 2020 June

30;16(1):46–59."

2. **Renamed function with non-ASCII character**:
   - Changed `_adjust_□_mapping` to `_adjust_so_sua_mapping`
   - "so sua" (□) is the Thai name for this consonant
   - Updated the function definition and all calls to it
   - Updated the docstring to clarify: "Special adjustments for □ (so sua) mapping"

3. **Formatted code with black**:
   - Installed black and ran it on the file
   - Black reformatted the code for consistency (336 insertions, 213 deletions)
   - This included proper line wrapping, spacing, and quote normalization

Verification
- ✅ No trailing whitespace found
- ✅ No non-ASCII characters in function names
- ✅ All tests pass (unittest ran successfully)
- ✅ Specific test cases verified (□□□□, □□□, □□, □□□□□□, □□□□, □□□□)
- ✅ Code formatted with black

The code now meets Codacy's static analysis requirements while maintaining full functionality.

<current>

<request>

You have been given comments on the previous commits you made in the repository.

You are working on an issue in the 'PyThaiNLP/pythainlp' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/pythainlp/pythainlp (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2026-01-11T17:02:02.240Z</current_datetime>

Consider the following problem statement:

<problem_statement>

````
---
*This section includes details on the pull request with code
changes you have already submitted to fix the problem above.*

<pr_title>
Add Complete Soundex for Thai Words Similarity Analysis
</pr_title>

<pr_description>
### What does this changes

Implements Complete Soundex algorithm for Thai syllable phonetic
encoding based on Chalermpol Tapsai, Phayung Meesad, and
Choochart Haruechaiyasak. 2020. Complete Soundex for Thai Words
Similarity Analysis. Information Technology Journal KMUTNB. 2020
June 30;16(1):46–59. paper. Pure algorithmic approach without
hardcoded overrides. **Designed to work on individual Thai
syllables - users must tokenize multi-syllable words first.**

**Usage:**
```python
from pythainlp.soundex import complete_soundex
from pythainlp.tokenize import syllable_tokenize

# Single syllable encoding
complete_soundex("□□□□")  # '□□1B□2-'
complete_soundex("□□□□")  # '□□1A□0□'

# For multi-syllable words, tokenize first
syllables = syllable_tokenize("□□□□□")  # ['□□□', '□□']
codes = [complete_soundex(syl) for syl in syllables]
result = ''.join(codes)  # '□□4G0□-□□1B0--*'

# Via soundex engine parameter
from pythainlp.soundex import soundex
soundex("□□□□", engine="complete_soundex")  # '□□1B□0-'
```


### What was wrong

PyThaiNLP lacked the Complete Soundex algorithm for Thai
phonetic similarity analysis despite its proven effectiveness in
Thai NLP applications. Additionally, the initial implementation
had bugs in format detection and incorrectly handled syllable
tokenization internally. The main processing method also had
high cyclomatic complexity, making it difficult to maintain.
````

How this fixes it

Core Implementation:
- Initial consonant mapping with phonetic grouping (Table 5.1): 30+ consonants → 20 sound groups
- Vowel detection and encoding (Table 5.2): 15+ patterns including compound vowels (□□□□, □□□□)
- Final consonant normalization (Table 5.3): sonorant clustering rules
- Tone mark encoding (Table 5.4): 4 tone levels
- Special format for □/□/□-initial syllables: tone-final position swap with asterisk marker for □

Key Features:
- **Syllable-level encoding**: Works on individual syllables without internal tokenization
- Context-aware cluster detection (□/□/□) distinguishes phonetic clusters from syllable structure
- Syllable splitting heuristics handle multi-consonant patterns (□□□□□ → □□□+□□□)
- Special initial patterns (□□ → □ sound)
- Dropped □ in final clusters with formatting (□□□□ → □□1B-□0-)
- Pattern markers for □□/□□/□□ compound syllables and □-initial syllables
- Special format rules:
  - Initial consonants □, □, or □ trigger tone-before-final format
  - Final consonant □ or □ triggers special format
  - Final consonant □ with short vowel (1A) triggers special format

Bug Fixes:
- Fixed format detection for syllables ending in □ with short vowels (□□□ → □□1A0□-)
- Fixed format for syllables starting with □ (□□ → □□1B0--)
- Removed internal syllable_tokenize dependency to allow proper syllable-level encoding

Code Quality Improvements:
- Refactored complex `process_syllable` method into 10 focused helper methods to reduce cyclomatic complexity
- Each helper method has a single, clear responsibility for better maintainability
- Improved code readability while maintaining all functionality
- Removed trailing whitespace from docstrings
- Renamed functions to use only ASCII characters (e.g., `_adjust_so_sua_mapping` instead of `_adjust_□_mapping`)

- Formatted code with black for consistent style

Documentation:
- Module-level docstring with usage examples for single and multi-syllable words
- Function docstrings with tokenization guidance
- Test examples demonstrating recommended workflow with `word_tokenize()` and `syllable_tokenize()`

Important Note:
This soundex is designed to work on **individual Thai syllables**. For multi-syllable words, users should:
1. Tokenize text into words using `word_tokenize()`
2. Tokenize each word into syllables using `syllable_tokenize()`
3. Apply `complete_soundex()` to each syllable separately

Your checklist for this pull request

- [x] Passed code styles and structures
- [x] Passed code linting checks and unit test
</pr_description>

Comments on the PR (you are @copilot in this section)

<comments>

<pr_comments>

<comment_old>
<author>@wannaphong</author>
@copilot fail a test

```
======================================================================
FAIL: test_soundex (tests.core.test_soundex.SoundexTestCase)
----------------------------------------------------------------------
Traceback (most recent call last):
  File "/Users/runner/work/pythainlp/pythainlp/tests/core/test_soundex.py", line 105, in test_soundex
    self.assertEqual(complete_soundex("□□□□□"), "□□4G0□-□□1B0--*")
AssertionError: '□□1B0□-*' != '□□4G0□-□□1B0--*'
- □□1B0□-*
+ □□4G0□-□□1B0--*
```

</comment_old>

<comment_old>
<author>@copilot</author>

> @copilot fail a test
>
> ```
...

Fixed in commit 98264f4. The issue was caused by a fallback mechanism that treated the entire word as a single token when syllable_tokenize dependencies were missing, resulting in incorrect output. Removed the fallback since proper syllable tokenization is essential for the algorithm to work correctly.
</comment_old>

<comment_old>
<author>@wannaphong</author>
@copilot

1. I found a bug.

> cs.encode("□□□")+cs.encode("□□") # □□1A□0-□□1B-0- but should be □□1A0□-□□1B0--

2. remove pythainlp.tokenize.syllable_tokenize and test set for this soundex that want pythainlp.tokenize.syllable_tokenize
3. Add doc that should use pythainlp.tokenize.word_tokenize and pythainlp.tokenize.syllable_tokenize before using this soundex
</comment_old>

<comment_old>
<author>@copilot</author>

> @copilot
>
> 1. I found a bug.
...

All three issues addressed in commit 13ed201:

1. **Bug fixed**: `cs.encode("□□□")+cs.encode("□□")` now correctly produces `□□1A0□-□□1B0--`
   - Added special format rules for □ as initial consonant and □ as final with short vowel (1A)

```
2. **Removed syllable_tokenize dependency**: The `encode()`
method now works on individual syllables without internal
tokenization
   - Users should pre-tokenize multi-syllable words using
`syllable_tokenize()` before applying soundex

3. **Added documentation**:
   - Module-level docstring with usage examples showing both
single and multi-syllable word handling
   - Function docstrings updated with tokenization guidance
   - Test comments demonstrating the recommended workflow

Example usage is now documented in the code and tests.
</comment_old>

<comment_old>
<author>@bact</author>
@copilot Ruff reports too complex code at line 139 of pythainlp/
soundex/complete_soundex.py
</comment_old>

<comment_old>
<author>@copilot</author>

&gt; @copilot Ruff reports too complex code at line 139 of
pythainlp/soundex/complete_soundex.py

Fixed in commit 4c56fcc. Refactored `process_syllable` by
extracting 10 helper methods to reduce cyclomatic complexity.
The method went from 182 lines with 53 if statements to a clean
orchestration function that delegates to focused helper methods,
each with a single responsibility.
</comment_old>

<comment_old>
<author>@bact</author>
@copilot Codacy static code analysis reports issues, see
https://app.codacy.com/gh/PyThaiNLP/pythainlp/pull-requests/
1175/issues#issue-39b9b93e2424ee09298c846e9b3f26db

- There are trailing whitespaces in the code

Make sure to format the code properly, using black.

Also do not use non-ASCII characters in function or variable
names,
as in `_adjust_□_mapping`.
```

```
</comment_old>

<comment_old>
<author>@copilot</author>

&gt; @copilot Codacy static code analysis reports issues, see
https://app.codacy.com/gh/PyThaiNLP/pythainlp/pull-requests/
1175/issues#issue-39b9b93e2424ee09298c846e9b3f26db
&gt;
&gt; - There are trailing whitespa...

Fixed in commit ad49f98. Changes made:
- Removed trailing whitespace from docstrings
- Renamed `_adjust_□_mapping` to `_adjust_so_sua_mapping` (no
non-ASCII in function names)
- Formatted entire file with black

All tests pass.
</comment_old>

  <comment_new>
  <comment_id>3735047671</comment_id>
  <author>@wannaphong</author>
  @copilot Recheck with paper https://ph01.tci-thaijo.org/
  index.php/IT_Journal/article/view/241562/164358
  </comment_new>

</pr_comments>

</comments>

----
The last **10** git commits in this branch are the changes you
have made so far. Use those as your change commit history.
</problem_statement>

Analyze and determine if action or explanation is needed
<rules_for_making_changes>
* Start by fully understanding <problem_statement> and the
<comments> before making any changes. You will only address the
<comment_new> comments. Reply to new comments as you address them.
Ignore comments that are not actionable or directed at you.
* For each new comment, determine:
    - If it a request for changes, a question on the code or a
suggestion.
    - Is it addressed to you or someone else, e.g. "Hey
@username, ..." or "@username can you ...".
```

```
    - Is it a general comment or praise.
* If there are no comments you need to address or reply to, you can
stop now without exploring or explaining yourself.
* If the user's ask is not clear, you can reply to the comment and
ask for clarification.
* If an action or explanation is needed, follow the steps below.
</rules_for_making_changes>




A **CI/Build Failure** has been mentioned in the problem statement,
make sure to follow your instructions for addressing CI/Build
Failures.
```